

API 规范：代码支撑平台

1. 通用约定

基础路径： `/api/v1`

认证规则：需要登录的接口通过 `Authorization` 请求头传递 JWT。

认证方式：

```
Authorization: Bearer <JWT_TOKEN>
```

模块定位：

GitHub 作为外部仓库导入与单向同步来源。

Gitea 作为平台内代码协作底座。

本模块只提供代码托管能力适配，不处理任务推荐、成长规则和业务审核决策。

不支持 Git Guild / Gitea 到 GitHub 的反向同步。

不提供网页代码编辑器，只支持上传变更生成 commit。

统一成功响应：

业务码： `SUCCESS`

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {},
  "timestamp": "yyyy-mm-ddThh:mm:ss+08:00",
  "traceId": "req-code-0001"
}
```

统一失败响应：

错误码： `CODE_HOST_UNAVAILABLE`

```
{
  "code": "CODE_HOST_UNAVAILABLE",
  "message": "代码托管平台暂不可用",
  "details": "Gitea API 请求超时",
  "timestamp": "yyyy-mm-ddThh:mm:ss+08:00",
  "traceId": "req-code-0002"
}
```

2. 业务码/错误码

业务码	HTTP 状态码	说明
SUCCESS	200 / 201 / 202 / 204	请求成功

错误码	HTTP 状态码	说明
VALIDATION_FAILED	400	参数缺失或格式错误
UNAUTHORIZED	401	未登录或令牌无效
FORBIDDEN	403	当前用户无权限
UNSUPPORTED_CODE_HOST	400	不支持的代码托管平台类型
REPOSITORY_NOT_FOUND	404	仓库不存在
ISSUE_NOT_FOUND	404	Issue 不存在
PULL_REQUEST_NOT_FOUND	404	Pull Request 不存在
BRANCH_NOT_FOUND	404	分支不存在
REPOSITORY_ALREADY_IMPORTED	409	仓库已导入
SYNC_IN_PROGRESS	409	仓库正在同步
BRANCH_ALREADY_EXISTS	409	分支已存在
COMMIT_REJECTED	409	提交被代码托管平台拒绝
PULL_REQUEST_ALREADY_EXISTS	409	Pull Request 已存在
WEBHOOK_SIGNATURE_INVALID	401	Webhook 签名校验失败
CODE_HOST_UNAVAILABLE	502	外部代码托管平台不可用
REPOSITORY_IMPORT_FAILED	502	仓库导入失败
INTERNAL_ERROR	500	服务器内部错误

3. 导入或接入仓库

接口： `POST /api/v1/repositories/import`

功能：从 GitHub 导入仓库元数据，或将已有仓库接入平台内 Gitea 协作底座。

权限：登录用户，角色为 `MAINTAINER` 或 `ADMIN`。

请求参数

请求头：

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>
Content-Type	String	是	application/json

请求体：

参数名	类型	必填	说明
hostType	String	是	GITHUB 或 GITEA
sourceUrl	String	是	外部仓库地址或 Gitea 仓库地址
owner	String	是	仓库所属组织或用户
name	String	是	仓库名称
visibility	String	否	PUBLIC 或 PRIVATE，默认 PUBLIC
syncIssues	Boolean	否	是否导入后立即同步 Issue，默认 true

请求示例：

```
POST /api/v1/repositories/import
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "hostType": "GITHUB",
  "sourceUrl": "https://github.com/example/git-guild-demo",
  "owner": "example",
  "name": "git-guild-demo",
  "visibility": "PUBLIC",
  "syncIssues": true
}
```

成功响应

HTTP 状态码： 201 Created

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "仓库已接入",
  "data": {
    "repositoryId": 1001,
    "name": "git-guild-demo",
    "sourceUrl": "https://github.com/example/git-guild-demo",
    "hostType": "GITHUB",
    "syncStatus": "SYNCING",
    "importedBy": 10001,
    "createdAt": "2026-05-15T20:30:00+08:00"
  },
  "timestamp": "2026-05-15T20:30:00+08:00",
  "traceId": "req-code-0003"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: REPOSITORY_ALREADY_IMPORTED

```
{
  "code": "REPOSITORY_ALREADY_IMPORTED",
  "message": "仓库已导入",
  "details": "该 sourceUrl 已存在对应仓库记录",
  "timestamp": "2026-05-15T20:30:00+08:00",
  "traceId": "req-code-0004"
}
```

4. 查看仓库详情

接口: GET /api/v1/repositories/{repositoryId}

功能: 查看仓库元数据、同步状态和平台内协作地址。

权限: 公开仓库可公开访问；私有仓库需要登录且具备权限。

请求参数

路径参数:

参数名	类型	必填	说明
repositoryId	Long	是	仓库 ID

请求头：

参数名	类型	必填	说明
Authorization	String	否	私有仓库必须携带登录令牌

请求示例：

```
GET /api/v1/repositories/1001
Authorization: Bearer <JWT_TOKEN>
```

成功响应

HTTP 状态码： 200 OK

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {
    "repositoryId": 1001,
    "name": "git-guild-demo",
    "owner": "example",
    "sourceUrl": "https://github.com/example/git-guild-demo",
    "hostType": "GITHUB",
    "giteaUrl": "https://gitea.gitguild.local/example/git-guild-demo",
    "visibility": "PUBLIC",
    "syncStatus": "SUCCESS",
    "lastSyncedAt": "2026-05-15T20:35:00+08:00"
  },
  "timestamp": "2026-05-15T20:36:00+08:00",
  "traceId": "req-code-0005"
}
```

失败响应

HTTP 状态码： 404 Not Found

错误码： REPOSITORY_NOT_FOUND

```
{
  "code": "REPOSITORY_NOT_FOUND",
  "message": "仓库不存在",
  "details": "repositoryId=1001 未找到",
  "timestamp": "2026-05-15T20:36:00+08:00",
  "traceId": "req-code-0006"
}
```

5. 手动同步仓库元数据

接口： `POST /api/v1/repositories/{repositoryId}/sync`

功能：手动触发仓库 Issue、PR、分支和 commit 元数据同步。

权限：登录用户，角色为 `MAINTAINER` 或 `ADMIN`。

请求参数

路径参数：

参数名	类型	必填	说明
<code>repositoryId</code>	<code>Long</code>	是	仓库 ID

请求头：

参数名	类型	必填	说明
<code>Authorization</code>	<code>String</code>	是	登录令牌，格式为 <code>Bearer <JWT_TOKEN></code>
<code>Content-Type</code>	<code>String</code>	是	<code>application/json</code>

请求体：

参数名	类型	必填	说明
<code>syncScope</code>	<code>Array<String></code>	否	同步范围，支持 <code>ISSUES</code> 、 <code>PULL_REQUESTS</code> 、 <code>BRANCHES</code> 、 <code>COMMITTS</code>

请求示例：

```
POST /api/v1/repositories/1001/sync
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

```
{
  "syncScope": ["ISSUES", "PULL_REQUESTS"]
}
```

成功响应

HTTP 状态码: 202 Accepted

业务码: SUCCESS

```
{
  "code": "SUCCESS",
  "message": "同步任务已创建",
  "data": {
    "repositoryId": 1001,
    "syncStatus": "SYNCING",
    "syncJobId": "sync-20260515-0001"
  },
  "timestamp": "2026-05-15T20:40:00+08:00",
  "traceId": "req-code-0007"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: SYNC_IN_PROGRESS

```
{
  "code": "SYNC_IN_PROGRESS",
  "message": "仓库正在同步",
  "details": "请等待当前同步任务完成后重试",
  "timestamp": "2026-05-15T20:40:00+08:00",
  "traceId": "req-code-0008"
}
```

6. 查询仓库 Issue 列表

接口: GET /api/v1/repositories/{repositoryId}/issues

功能：查询仓库 Issue 元数据，供任务发布时选择来源 Issue。

权限：登录用户，角色为 `MAINTAINER` 或 `ADMIN`。

请求参数

路径参数：

参数名	类型	必填	说明
<code>repositoryId</code>	<code>Long</code>	是	仓库 ID

请求头：

参数名	类型	必填	说明
<code>Authorization</code>	<code>String</code>	是	登录令牌，格式为 <code>Bearer <JWT_TOKEN></code>

Query 参数：

参数名	类型	必填	说明
<code>status</code>	<code>String</code>	否	Issue 状态， <code>OPEN</code> 、 <code>CLOSED</code>
<code>keyword</code>	<code>String</code>	否	标题关键词
<code>page</code>	<code>Integer</code>	否	页码，从 <code>1</code> 开始，默认 <code>1</code>
<code>size</code>	<code>Integer</code>	否	每页数量，默认 <code>20</code> ，最大 <code>100</code>

请求示例：

```
GET /api/v1/repositories/1001/issues?status=OPEN&page=1&size=20
Authorization: Bearer <JWT_TOKEN>
```

成功响应

HTTP 状态码： `200 OK`

业务码： `SUCCESS`

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {
    "items": [
      {
        "issueId": 3001,
        "externalIssueId": "42",
        "title": "补充 Issue 同步状态页面",
        "status": "OPEN",
        "canCreateQuest": true,
        "updatedAt": "2026-05-15T19:30:00+08:00"
      }
    ],
    "page": 1,
    "size": 20,
    "total": 1
  },
  "timestamp": "2026-05-15T20:45:00+08:00",
  "traceId": "req-code-0009"
}
```

失败响应

HTTP 状态码: 404 Not Found

错误码: REPOSITORY_NOT_FOUND

```
{
  "code": "REPOSITORY_NOT_FOUND",
  "message": "仓库不存在",
  "details": "repositoryId=1001 未找到",
  "timestamp": "2026-05-15T20:45:00+08:00",
  "traceId": "req-code-0010"
}
```

7. 创建开发分支

接口: POST /api/v1/repositories/{repositoryId}/branches

功能: 在平台内 Gitea 仓库中基于指定分支创建开发分支。

权限: 登录用户, 且用户已接取与该仓库相关的任务, 或角色为 MAINTAINER / ADMIN。

请求参数

路径参数：

参数名	类型	必填	说明
repositoryId	Long	是	仓库 ID

请求头：

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>
Content-Type	String	是	application/json

请求体：

参数名	类型	必填	说明
baseBranch	String	是	基准分支，例如 main
newBranch	String	是	新分支名称，例如 quest-5001
questId	Long	否	关联任务 ID

请求示例：

```
POST /api/v1/repositories/1001/branches
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "baseBranch": "main",
  "newBranch": "quest-5001",
  "questId": 5001
}
```

成功响应

HTTP 状态码： 201 Created

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "分支已创建",
  "data": {
    "repositoryId": 1001,
    "branchName": "quest-5001",
    "baseBranch": "main",
    "createdAt": "2026-05-15T20:50:00+08:00"
  },
  "timestamp": "2026-05-15T20:50:00+08:00",
  "traceId": "req-code-0011"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: BRANCH_ALREADY_EXISTS

```
{
  "code": "BRANCH_ALREADY_EXISTS",
  "message": "分支已存在",
  "details": "quest-5001 已存在",
  "timestamp": "2026-05-15T20:50:00+08:00",
  "traceId": "req-code-0012"
}
```

8. 上传变更并生成 Commit

接口: POST /api/v1/repositories/{repositoryId}/commits

功能: 用户上传文件变更, 由平台调用 Gitea API 生成 commit。

权限: 登录用户, 且用户已接取与该仓库相关的任务, 或角色为 MAINTAINER / ADMIN。

请求参数

路径参数:

参数名	类型	必填	说明
repositoryId	Long	是	仓库 ID

请求头:

--	--	--	--

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>
Content-Type	String	是	application/json

请求体：

参数名	类型	必填	说明
branch	String	是	目标分支
message	String	是	commit message，长度 1-200
questId	Long	否	关联任务 ID
changes	Array<Object>	是	文件变更列表

changes 子项：

参数名	类型	必填	说明
path	String	是	文件路径
action	String	是	CREATE 、 UPDATE 、 DELETE
content	String	否	Base64 编码后的文件内容，删除时可为空

请求示例：

```
POST /api/v1/repositories/1001/commits
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

```
{
  "branch": "quest-5001",
  "message": "feat: add issue sync status view",
  "questId": 5001,
  "changes": [
    {
      "path": "src/views/IssueSyncStatus.vue",
      "action": "CREATE",
      "content": "PGh0bWw+Li4uPC9odG1sPg=="
    }
  ]
}
```

成功响应

HTTP 状态码: 201 Created

业务码: SUCCESS

```
{
  "code": "SUCCESS",
  "message": "Commit 已创建",
  "data": {
    "repositoryId": 1001,
    "branch": "quest-5001",
    "commitSha": "9fceb02",
    "commitUrl": "https://gitea.gitguild.local/example/git-guild-demo/commit/9fceb02",
    "createdAt": "2026-05-15T21:00:00+08:00"
  },
  "timestamp": "2026-05-15T21:00:00+08:00",
  "traceId": "req-code-0013"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: COMMIT_REJECTED

```
{
  "code": "COMMIT_REJECTED",
  "message": "Commit 被拒绝",
  "details": "目标分支不存在或文件版本冲突",
  "timestamp": "2026-05-15T21:00:00+08:00",
  "traceId": "req-code-0014"
}
```

9. 创建 Pull Request

接口： `POST /api/v1/repositories/{repositoryId}/pull-requests`

功能：基于开发分支向目标分支发起 Pull Request。

权限：登录用户，且用户已接取与该仓库相关的任务，或角色为 `MAINTAINER` / `ADMIN`。

请求参数

路径参数：

参数名	类型	必填	说明
<code>repositoryId</code>	<code>Long</code>	是	仓库 ID

请求头：

参数名	类型	必填	说明
<code>Authorization</code>	<code>String</code>	是	登录令牌，格式为 <code>Bearer <JWT_TOKEN></code>
<code>Content-Type</code>	<code>String</code>	是	<code>application/json</code>

请求体：

参数名	类型	必填	说明
sourceBranch	String	是	源分支
targetBranch	String	是	目标分支
title	String	是	PR 标题
description	String	否	PR 描述
questId	Long	否	关联任务 ID

请求示例：

```
POST /api/v1/repositories/1001/pull-requests
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

```
{
  "sourceBranch": "quest-5001",
  "targetBranch": "main",
  "title": "实现 Issue 同步状态页面",
  "description": "关联任务 #5001，补充同步状态展示。",
  "questId": 5001
}
```

成功响应

HTTP 状态码： 201 Created

业务码： SUCCESS


```
{
  "code": "SUCCESS",
  "message": "Pull Request 已创建",
  "data": {
    "pullRequestId": 7001,
    "externalPrId": "128",
    "title": "实现 Issue 同步状态页面",
    "status": "OPEN",
    "url": "https://gitea.gitguild.local/example/git-guild-demo/pulls/128",
    "createdAt": "2026-05-15T21:10:00+08:00"
  },
  "timestamp": "2026-05-15T21:10:00+08:00",
  "traceId": "req-code-0015"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: PULL_REQUEST_ALREADY_EXISTS

```
{
  "code": "PULL_REQUEST_ALREADY_EXISTS",
  "message": "Pull Request 已存在",
  "details": "sourceBranch=quest-5001 已存在打开状态的 PR",
  "timestamp": "2026-05-15T21:10:00+08:00",
  "traceId": "req-code-0016"
}
```

10. 查看 Pull Request 状态

接口: GET /api/v1/repositories/{repositoryId}/pull-requests/{pullRequestId}

功能: 查询 Pull Request 状态, 供任务提交和审核流程引用。

权限: 登录用户。公开仓库可放宽为公开访问。

请求参数

路径参数:

参数名	类型	必填	说明
repositoryId	Long	是	仓库 ID
pullRequestId	Long	是	PR ID

请求头：

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>

请求示例：

```
GET /api/v1/repositories/1001/pull-requests/7001
Authorization: Bearer <JWT_TOKEN>
```

成功响应

HTTP 状态码： 200 OK

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {
    "pullRequestId": 7001,
    "externalPrId": "128",
    "repositoryId": 1001,
    "title": "实现 Issue 同步状态页面",
    "status": "OPEN",
    "sourceBranch": "quest-5001",
    "targetBranch": "main",
    "url": "https://gitea.gitguild.local/example/git-guild-demo/pulls/128",
    "updatedAt": "2026-05-15T21:20:00+08:00"
  },
  "timestamp": "2026-05-15T21:20:00+08:00",
  "traceId": "req-code-0017"
}
```

失败响应

HTTP 状态码： 404 Not Found

错误码： `PULL_REQUEST_NOT_FOUND`

```
{
  "code": "PULL_REQUEST_NOT_FOUND",
  "message": "Pull Request 不存在",
  "details": "pullRequestId=7001 未找到",
  "timestamp": "2026-05-15T21:20:00+08:00",
  "traceId": "req-code-0018"
}
```

11. 接收代码托管 Webhook

接口： `POST /api/v1/code-host/webhooks/{hostType}`

功能：接收 GitHub 或 Gitea 的 Webhook 事件，并转换为平台内部事件。

权限：外部平台签名校验通过。

请求参数

路径参数：

参数名	类型	必填	说明
<code>hostType</code>	<code>String</code>	是	<code>GITHUB</code> 或 <code>GITEA</code>

请求头：

参数名	类型	必填	说明
<code>X-GitGuild-Signature</code>	<code>String</code>	是	Webhook 签名
<code>X-GitGuild-Event</code>	<code>String</code>	是	事件类型
<code>Content-Type</code>	<code>String</code>	是	<code>application/json</code>

请求体：由外部代码托管平台推送，服务端按 `hostType` 选择对应适配器解析。

请求示例：

```
POST /api/v1/code-host/webhooks/GITEA
X-GitGuild-Signature: sha256=xxx
X-GitGuild-Event: pull_request
Content-Type: application/json
```

成功响应

HTTP 状态码: 204 No Content

业务码: SUCCESS

响应体为空。

失败响应

HTTP 状态码: 401 Unauthorized

错误码: WEBHOOK_SIGNATURE_INVALID

```
{
  "code": "WEBHOOK_SIGNATURE_INVALID",
  "message": "Webhook 签名校验失败",
  "details": "签名与请求体摘要不匹配",
  "timestamp": "2026-05-15T21:30:00+08:00",
  "traceId": "req-code-0019"
}
```